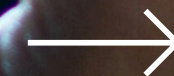


TIDY01: Fundamentals of Tidyverse I

Presented by Gabriel Rodrigues Palma



Tidyverse for Ecologists
(TIDY01)



Day 2 (9:30 – 17:30)

Data Cleaning
(9:35 – 11:05)



Tidyverse for Ecologists
(TIDY01)



Day 2 (9:30 – 17:30)

Data Cleaning
(9:35 – 11:05)

Data Filtering and
Selecting (11:05 – 12:30)

Tidyverse for Ecologists
(TIDY01)



Day 2 (9:30 – 17:30)

Data Cleaning
(9:35 – 11:05)

Grammar of Graphics
(13:30 – 15:00)

Data Filtering and
Selecting (11:05 – 12:30)

Tidyverse for Ecologists
(TIDY01)



Day 2 (9:30 – 17:30)

Data Cleaning
(9:35 – 11:05)

Grammar of Graphics
(13:30 – 15:00)

Data Filtering and
Selecting (11:05 – 12:30)

Basic Plots
(15:00 – 16:30)

Tidyverse for Ecologists
(TIDY01)



Day 2 (9:30 – 17:30)

Data Cleaning
(9:35 – 11:05)

Grammar of Graphics
(13:30 – 15:00)

Layers, Scales, and
Faceting (16:30 – 17:30)

Data Filtering and
Selecting (11:05 – 12:30)

Basic Plots
(15:00 – 16:30)

Tidyverse for Ecologists
(TIDY01)



Data Cleaning

Tidyverse for Ecologists
(TIDY01)

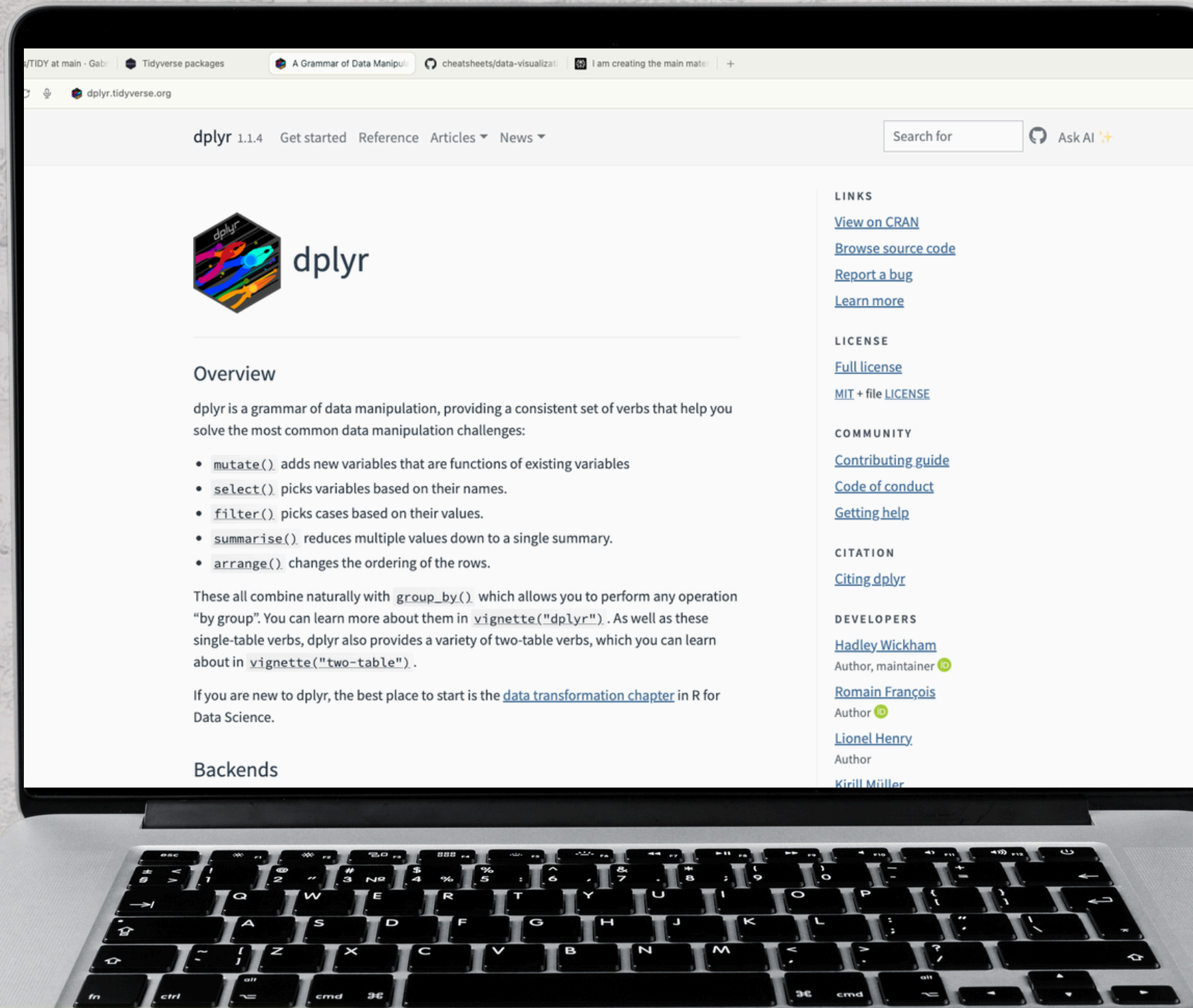


Data Cleaning

Tidyverse for Ecologists
(TIDY01)



Data Cleaning



Tidyverse for Ecologists
(TIDY01)



Data Cleaning

Data transformation with dplyr : : CHEATSHEET



dplyr functions work with pipes and expect **tidy data**. In tidy data:



Each **variable** is in its own **column**



Each **observation**, or **case**, is in its own **row**

pipes

x |> f(y) becomes **f(x, y)**

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function



summarize(.data, ...)
Compute table of summaries.
mtcars |> summarize(avg = mean(mpg))



count(.data, ..., wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in ... Also **tally()**, **add_count()**, **add_tally()**.
mtcars |> count(cyl)

Group Cases

Use **group_by(.data, ..., add = FALSE, drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



mtcars |>
group_by(cyl) |>
summarize(avg = mean(mpg))

Use **rowwise(.data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidyR cheat sheet for list-column workflow.



starwars |>
rowwise() |>
mutate(film_count = length(films))

ungroup(x, ...) Returns ungrouped copy of table.
g_mtcars <- mtcars |> group_by(cyl)
ungroup(g_mtcars)

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.



filter(.data, ..., .preserve = FALSE) Extract rows that meet logical criteria.
mtcars |> filter(mpg > 20)



distinct(.data, ..., .keep_all = FALSE) Remove rows with duplicate values.
mtcars |> distinct(gear)



slice(.data, ..., .preserve = FALSE) Select rows by position.
mtcars |> slice(10:15)



slice_sample(.data, ..., n, prop, weight_by = NULL, replace = FALSE) Randomly select rows. Use **n** to select a number of rows and **prop** to select a fraction of rows.
mtcars |> slice_sample(n = 5, replace = TRUE)



slice_min(.data, order_by, ..., n, prop, with_ties = TRUE) and **slice_max()** Select rows with the lowest and highest values.
mtcars |> slice_min(mpg, prop = 0.25)



slice_head(.data, ..., n, prop) and **slice_tail()** Select the first or last rows.
mtcars |> slice_head(n = 5)

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%		xor()
!=	>	>=	!is.na()	!	&	

See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES



arrange(.data, ..., .by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
mtcars |> arrange(mpg)
mtcars |> arrange(desc(mpg))

ADD CASES



add_row(.data, ..., .before = NULL, .after = NULL) Add one or more rows to a table.
cars |> add_row(speed = 1, dist = 1)

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



pull(.data, var = -1, name = NULL, ...) Extract column values as a vector, by name or index.
mtcars |> pull(wt)



select(.data, ...) Extract columns as a table.
mtcars |> select(mpg, wt)



relocate(.data, ..., .before = NULL, .after = NULL) Move columns to new position.
mtcars |> relocate(mpg, cyl, .after = last_col())

Use these helpers with select() and across()

e.g. mtcars |> select(mpg:cyl)

contains(match)	num_range(prefix, range)	; e.g., mpg:cyl
ends_with(match)	all_of(x)/any_of(x, ..., vars)	! e.g., !gear
starts_with(match)	matches(match)	everything()

MANIPULATE MULTIPLE VARIABLES AT ONCE

df <- tibble(x_1 = c(1, 2), x_2 = c(3, 4), y = c(4, 5))



across(.cols, .funs, ..., .names = NULL) Summarize or mutate multiple columns in the same way.
df |> summarize(across(everything(), mean))



c_across(.cols) Compute across columns in row-wise data.
df |>
rowwise() |>
mutate(x_total = sum(c_across(1:2)))

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

vectorized function



mutate(.data, ..., .keep = "all", .before = NULL, .after = NULL) Compute new column(s). Also **add_column()**.
mtcars |> mutate(gpm = 1 / mpg)
mtcars |> mutate(gpm = 1 / mpg, .keep = "none")



rename(.data, ...) Rename columns. Use **rename_with()** to rename with a function.
mtcars |> rename(miles_per_gallon = mpg)



CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at dplyr.tidyverse.org • HTML cheatsheets at posit.co/cheatsheets • dplyr 1.1.4 • Updated: 2024-05

Tidyverse for Ecologists
(TIDY01)



Data Cleaning

Data transformation with dplyr :: CHEATSHEET

dplyr functions work with pipes and expect **tidy data**. In tidy data:



Each **variable** is in its own **column**



Each **observation**, or **case**, is in its own **row**

pipes

$x \mid> f(y)$ becomes $f(x, y)$

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function



summarize(.data, ...)
Compute table of summaries.
`mtcars > summarize(avg = mean(mpg))`



count(.data, ..., wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in ... Also **tally()**, **add_count()**, **add_tally()**.
`mtcars > count(cyl)`

Group Cases

Use **group_by(.data, ..., .add = FALSE, .drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



`mtcars > group_by(cyl) > summarize(avg = mean(mpg))`

Use **rowwise(.data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidy cheat sheet for list-column workflow.



`starwars > rowwise() > mutate(film_count = length(films))`

ungroup(x, ...) Returns ungrouped copy of table.
`g_mtcars <- mtcars > group_by(cyl)`
`ungroup(g_mtcars)`

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.



filter(.data, ..., .preserve = FALSE) Filter rows that meet logical criteria.
`mtcars > filter(mpg > 20)`



distinct(.data, ..., .keep = TRUE) Keep rows with duplicate values.
`mtcars > distinct(gear)`



slice(.data, ..., .preserve = TRUE) Slice by position.
`mtcars > slice(10:15)`



slice_sample(.data, ..., size = NULL, replace = FALSE, n = NULL) Use n to select a number of rows, or a fraction of rows.
`mtcars > slice_sample(n = 5)`



slice_min(.data, ..., by = NULL, n = 1, ties = TRUE) Select the row with the lowest values.
`mtcars > slice_min(mpg)`



slice_head(.data, ..., n = 1) Select the first or last n rows.
`mtcars > slice_head(n = 5)`

Logical and boolean operators to use with dplyr

Operator	Symbol
Equality	<code>==</code>
Less than	<code><</code>
Less than or equal to	<code><=</code>
Not equal to	<code>!=</code>
Greater than	<code>></code>
Greater than or equal to	<code>>=</code>
Is NA	<code>is.na()</code>
Not NA	<code>!is.na()</code>

See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES



arrange(.data, ..., .by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
`mtcars > arrange(mpg)`
`mtcars > arrange(desc(mpg))`

ADD CASES



add_row(.data, ..., .before = NULL, .after = NULL) Add one or more rows to a table.
`cars > add_row(speed = 1, dist = 1)`

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function

summarize(.data, ...)
Compute table of summaries.
`mtcars > summarize(avg = mean(mpg))`

count(.data, ..., wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in ... Also **tally()**, **add_count()**, **add_tally()**.
`mtcars > count(cyl)`

Group Cases

Use **group_by(.data, ..., .add = FALSE, .drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.

`mtcars > group_by(cyl) > summarize(avg = mean(mpg))`

`mtcars > group_by(cyl, mpg, .keep = "none")`

rename(.data, ...) Rename columns. Use **rename_with()** to rename with a function.
`mtcars > rename(miles_per_gallon = mpg)`



CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at dplyr.tidyverse.org • HTML cheatsheets at posit.co/cheatsheets • dplyr 1.1.4 • Updated: 2024-05

Tidyverse for Ecologists
(TIDY01)



Data Cleaning

R Input

```
# Soil nutrient classification
soil <- data.frame(
  nitrogen = c(38, 45, 62),
  plot = c("P1", "P2", "P3")
)
soil %>% mutate(n_level = case_when(
  nitrogen < 40 ~ "Low", nitrogen < 55 ~ "Medium", TRUE ~ "High"))
```



Console Output

	nitrogen	plot	n_level
1	38	P1	Low
2	45	P2	Medium
3	62	P3	High

Data Cleaning

R Input

```
# Water quality classification
water <- data.frame(
  pH = c(6.2, 7.1, 8.3),
  station = c("A", "B", "C")
)
water %>% mutate(pH_class = case_when(
  pH < 6.5 ~ "Acidic", pH > 7.5 ~ "Alkaline", TRUE ~ "Neutral"))
```



Console Output

	pH	station	pH_class
1	6.2	A	Acidic
2	7.1	B	Neutral
3	8.3	C	Alkaline



Data Cleaning

R Input

```
# Phenology processing
phenology <- data.frame(
  leaf_out = c(110, 125, 130),
  leaf_drop = c(280, 290, 285)
)
phenology %>% mutate(growing_season = leaf_drop - leaf_out)
```



Console Output

	leaf_out	leaf_drop	growing_season
1	110	280	170
2	125	290	165
3	130	285	155



Data Cleaning

R Input

```
# Bird survey data cleaning
library(dplyr)
birds <- data.frame(
  species = c("turdus migratorius", "PARUS MAJOR"),
  count = c(5, 3)
)
birds %>% mutate(species = stringr::str_to_title(species))
```



Console Output

```
species count
1 Turdus Migratorius    5
2 Parus Major           3
```


Data Cleaning

R Input

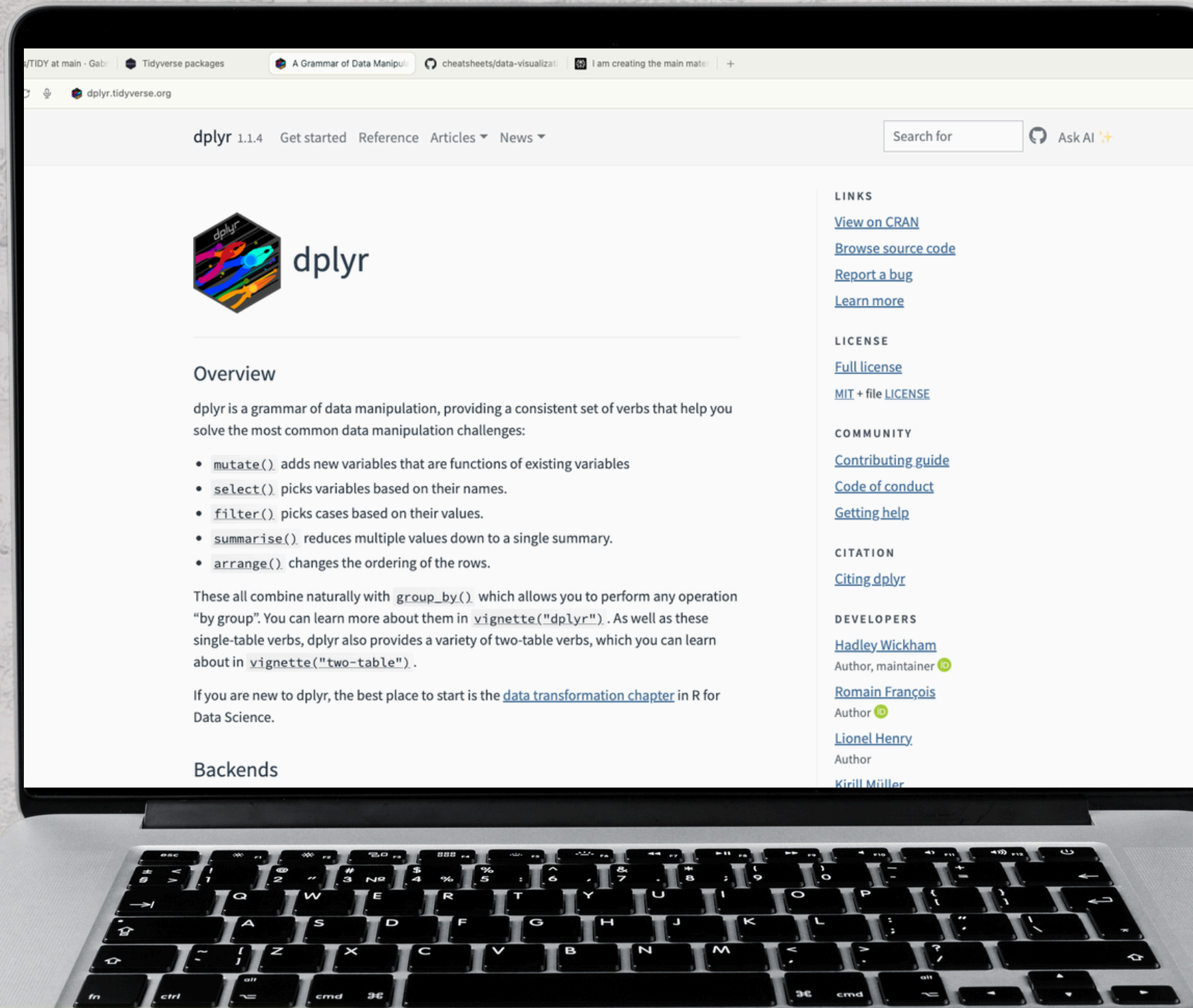
```
# Forest inventory calculations
forest <- data.frame(
  dbh_cm = c(15.2, 22.1),
  height_m = c(8.5, 12.3)
)
forest %>% mutate(basal_area = pi * (dbh_cm/200)^2)
```



Console Output

```
  dbh_cm height_m basal_area
1   15.2      8.5 0.01814760
2   22.1     12.3 0.03835029
```


Data Cleaning



Tidyverse for Ecologists
(TIDY01)



Day 2 (9:30 – 17:30)

Data Cleaning
(9:35 – 11:05)

Data Filtering and
Selecting (11:05 – 12:30)

Tidyverse for Ecologists
(TIDY01)



Data Filtering and Selecting

Tidyverse for Ecologists
(TIDY01)



Data Filtering and Selecting

Tidyverse for Ecologists
(TIDY01)



Data Filtering and Selecting

R Input

```
# Pollinator network summary
pollination <- data.frame(
  site = c('Urban', 'Rural', 'Urban'),
  visits = c(45, 62, 38)
)
pollination %>% group_by(site) %>%
  summarise(total_visits = sum(visits))
```



Console Output

```
# A tibble: 2 × 2
  site total_visits
<chr>      <dbl>
1 Rural         62
2 Urban         83
```


Data Filtering and Selecting

R Input

```
# Forest growth analysis
forest <- data.frame(
  species = c('Oak', 'Pine', 'Oak'),
  growth = c(2.5, 3.1, 2.8)
)
forest %>% group_by(species) %>%
  summarise(mean_growth = mean(growth))
```



Console Output

```
# A tibble: 2 × 2
  species mean_growth
  <chr>      <dbl>
1 Oak         2.65
2 Pine         3.1
```


Data Filtering and Selecting

R Input

```
# Community summary statistics
community <- data.frame(
  site = c('A', 'A', 'B', 'B'),
  abundance = c(10, 15, 8, 12)
)
community %>% group_by(site) %>%
  summarise(total = sum(abundance))
```



Console Output

```
# A tibble: 2 × 2
  site total
<chr> <dbl>
1 A      25
2 B      20
```


Data Filtering and Selecting

R Input

```
# Species abundance filtering
species_df <- data.frame(
  species = c('Oak', 'Pine', 'Maple'),
  count = c(25, 15, 8),
  dbh = c(20, 12, 18)
)
species_df %>% filter(count > 10, dbh >= 15)
```



Console Output

	species	count	dbh
1	Oak	25	20
2	Maple	8	18

Data Filtering and Selecting

R Input

```
# Water quality parameter selection
water <- data.frame(
  pH = c(6.8, 7.2, 8.1),
  temp = c(15, 18, 22),
  oxygen = c(8.5, 9.2, 7.8)
)
water %>% select(pH, oxygen) %>% filter(pH > 7.0)
```

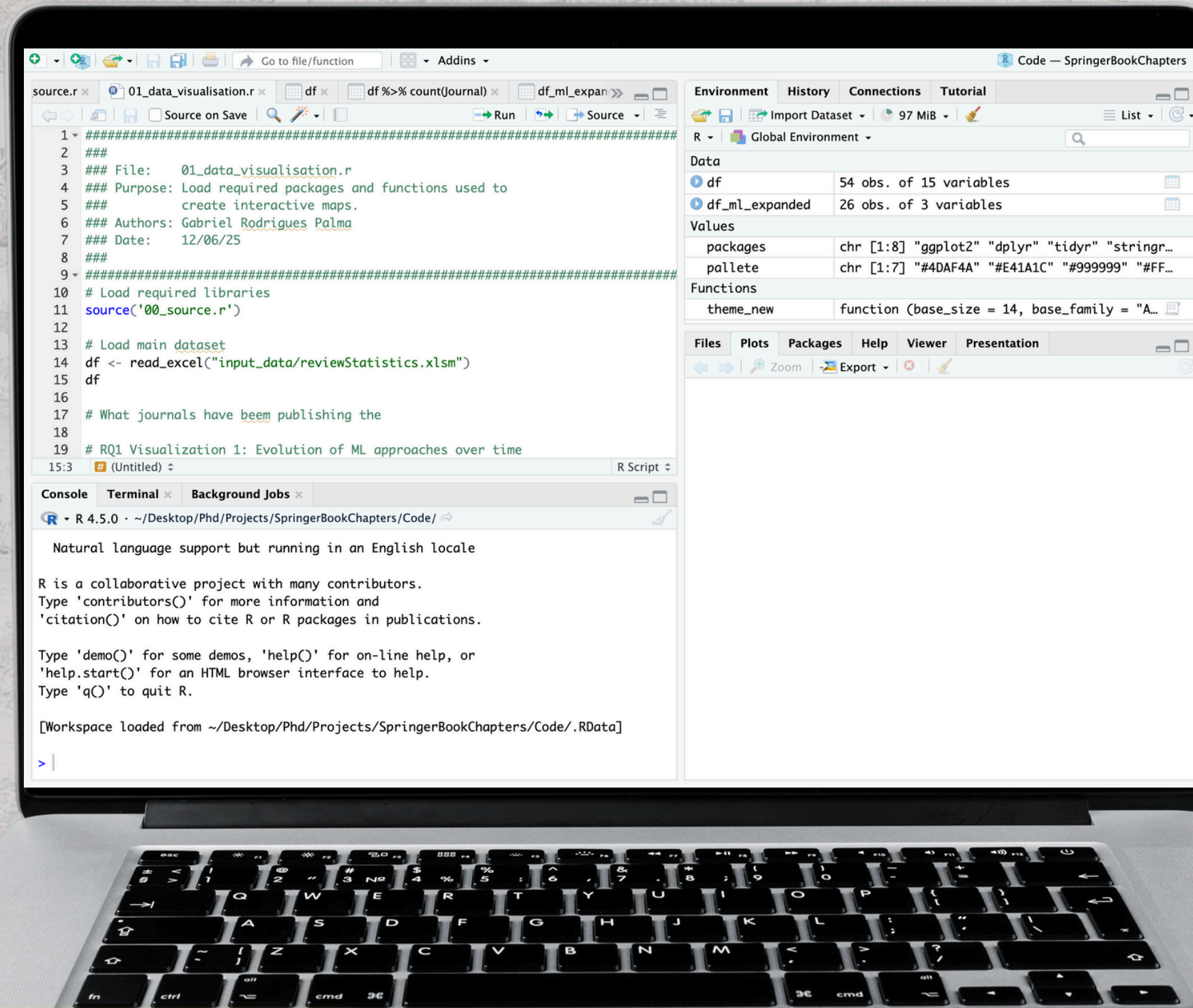


Console Output

	pH	oxygen
1	7.2	9.2
2	8.1	7.8



Data Filtering and Selecting



Tidyverse for Ecologists
(TIDY01)



Lunch time



Tidyverse for Ecologists
(TIDY01)

 stats

